Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 4:
Study and empirical analysis of dynamic programming algorithms.

Author:
st. gr. FAF-242 Temciuc Adelina
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2026

Contents

ALGORITHM ANALYSIS **2**
Objective 2
Tasks 2
Theoretical Notes 2
Introduction 3
Comparison Metric 3
Input Format 3

IMPLEMENTATION **4**
Dijkstra's Algorithm 4
Floyd-Warshall Algorithm 6

Comprehensive Comparison **8**
Dijkstra vs Floyd-Warshall — Sparse Graphs 8
Dijkstra vs Floyd-Warshall — Dense Graphs 9
Key Observations 10

Conclusion **11**

ALGORITHM ANALYSIS

Objective

Study and analyze the performance of dynamic programming-based shortest path algorithms — Dijkstra's Algorithm and Floyd-Warshall Algorithm — through empirical evaluation on graphs of varying size and density.

Tasks:

- Implement Dijkstra's Algorithm and Floyd-Warshall Algorithm in Python;
- Determine properties of input data (graphs) for algorithm analysis;
- Choose metrics for comparing the algorithms;
- Perform empirical analysis of the algorithms on sparse and dense graphs;
- Present the obtained results graphically;
- Draw conclusions from the laboratory work.

Theoretical Notes:

Dynamic Programming (DP) is a problem-solving paradigm that breaks a complex problem into overlapping sub-problems, solves each sub-problem once, and stores the results to avoid redundant computation. Unlike divide-and-conquer, which assumes independent sub-problems, DP exploits the optimal substructure property: an optimal solution to the overall problem incorporates optimal solutions to its sub-problems.

The two algorithms studied in this laboratory work both solve shortest path problems and are grounded in dynamic programming principles:

- **Dijkstra's Algorithm** — a greedy, single-source shortest path algorithm for graphs with non-negative edge weights. It maintains a priority queue to always expand the closest unvisited vertex. Using a binary min-heap, its time complexity is $O((V + E) \log V)$, where V is the number of vertices and E is the number of edges. It applies a DP-like relaxation: once a vertex's shortest distance is finalized, it is never reconsidered.

- **Floyd-Warshall Algorithm** — an all-pairs shortest path algorithm that computes shortest distances between every pair of vertices. It uses a three-dimensional DP recurrence:

$$\text{dist}[i][j][k] = \min(\text{dist}[i][j][k-1],\ \text{dist}[i][k][k-1] + \text{dist}[k][j][k-1])$$

where k is the intermediate vertex index. Its time complexity is $O(V^3)$ and space complexity is $O(V^2)$.

The key difference between the two algorithms lies in their scope and scalability. Dijkstra solves the single-source problem efficiently for sparse graphs, while Floyd-Warshall solves all-pairs shortest paths but scales poorly beyond a few hundred vertices due to its cubic complexity.

Introduction:

Shortest path problems are among the most practically important problems in computer science, with applications in routing protocols, geographic navigation, network analysis, and operations research. Two canonical dynamic programming solutions are studied here:

Dijkstra's algorithm is efficient for sparse graphs and excels when only a single source is needed. Floyd-Warshall is preferred when all-pairs distances are required and the graph is small enough to accommodate $O(V^3)$ computation.

While both algorithms are theoretically well understood, their empirical performance depends on implementation details, graph representation, and hardware characteristics. This laboratory work measures actual runtimes on randomly generated graphs under controlled conditions to quantify these practical differences.

Comparison Metric:

The comparison metric used in this laboratory work is execution time $T(n)$ measured in milliseconds. The `time.perf_counter()` function in Python provides high-resolution wall-clock timing suitable for short-duration measurements.

Each experiment is repeated 5 times per graph configuration, and the average execution time is computed to reduce noise from system fluctuations. The experimental error margin is set at 2.5 milliseconds.

Input Format:

The input data consists of randomly generated undirected, weighted graphs with varying structural properties:

- **Number of vertices:** 10, 50, 100, 200, 300, 500;
- **Edge weights:** uniformly random integers in $[1, 100]$ (non-negative, ensuring correctness of Dijkstra);
- **Graph density:** sparse ($\approx 20\%$ of possible edges) and dense ($\approx 80\%$ of possible edges);

- **Graph type:** undirected (edges added symmetrically), connected (guaranteed by a spanning tree seed);
- **Representation for Dijkstra:** adjacency list;
- **Representation for Floyd-Warshall:** adjacency matrix.

Each algorithm is tested with multiple randomly generated graphs per configuration, producing averaged runtime measurements.

IMPLEMENTATION

Both Dijkstra's and Floyd-Warshall algorithms are implemented in Python and evaluated experimentally by measuring their execution time across multiple graph configurations. The experimental error margin is set at 2.5 milliseconds.

Dijkstra's Algorithm:

Dijkstra's algorithm finds the shortest path from a single source vertex to all other vertices in a weighted graph with non-negative edge weights. It maintains a min-heap priority queue to greedily select the nearest unvisited vertex at each step, applying edge relaxation to update distances. The algorithm embodies dynamic programming in its systematic relaxation strategy: once a shortest path to a vertex is established, it is optimal and never revisited.

Time complexity: $O((V+E)\log V)$ with a binary heap. Space complexity: $O(V+E)$ for the adjacency list and heap.

Algorithm Overview:

Listing 1: Dijkstra's Algorithm (pseudocode)

```
Dijkstra(adj, src):
    dist[0..n-1] = INF
    dist[src] = 0
    heap = min-heap containing (0, src)

    while heap is not empty:
        d, u = heap.pop_min()
        if d > dist[u]:
            continue          // outdated entry, skip
        for each (v, w) in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heap.push((dist[v], v))

    return dist
```

Python Implementation:

Listing 2: Dijkstra's Algorithm in Python

```python
import heapq

INF = float('inf')

def dijkstra(adj, src):
    n = len(adj)
    dist = [INF] * n
    dist[src] = 0
    heap = [(0, src)]        # (distance, node)

    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue         # outdated entry
        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist
```

Results:

After executing Dijkstra's algorithm for each specified graph size and density, the following results were obtained:

Dijkstra's Algorithm – Results

n	Sparse (ms)	Dense (ms)
10	0.0140	0.0113
50	0.0666	0.1783
100	0.3301	0.6470
200	0.9323	1.9773
300	1.6201	6.3341
500	5.6148	37.4806

Figure 1: Dijkstra's Algorithm Performance Results (averaged over 5 runs)

The experimental data confirms the theoretical time complexity of $O((V+E)\log V)$ for Dijkstra. Sparse graphs grow gradually since fewer edges are relaxed per vertex. Dense graphs show a steeper increase because the heap processes many more edge relaxations.

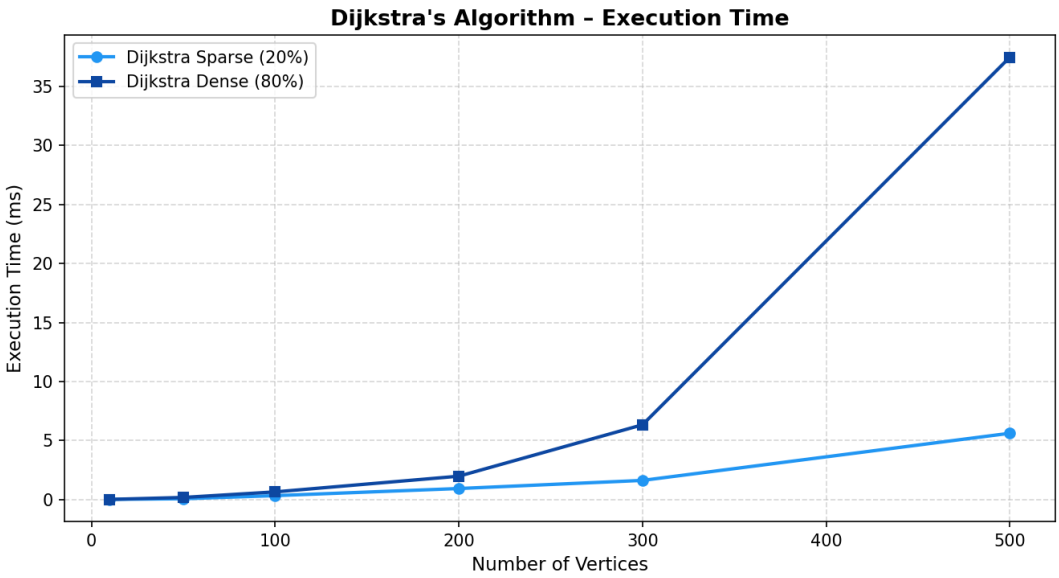

Figure 2: Dijkstra's Algorithm Execution Time vs Number of Vertices

Figure 2 illustrates the divergence between sparse and dense execution times as n grows. On dense graphs, the edge count scales quadratically ($E \approx 0.8 \cdot \frac{V(V-1)}{2}$), dramatically increasing heap operations. The iterative heap-based implementation avoids recursion overhead and performs efficiently even for larger inputs.

Floyd-Warshall Algorithm:

Floyd-Warshall computes shortest paths between all pairs of vertices simultaneously. Its core DP recurrence iterates over all possible intermediate vertices k and updates the distance matrix: if routing through k yields a shorter path from i to j, the matrix entry is updated. This directly embodies the DP principle of building optimal solutions from optimal sub-solutions.

Time complexity: $O(V^3)$. Space complexity: $O(V^2)$. Unlike Dijkstra, Floyd-Warshall is not sensitive to graph density — it always processes all V^3 combinations of intermediate vertices.

Algorithm Overview:

Listing 3: Floyd-Warshall Algorithm (pseudocode)

```
FloydWarshall(mat):
    dist = copy of mat
    for k from 0 to n-1:
        for i from 0 to n-1:
            for j from 0 to n-1:
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

Python Implementation:

Listing 4: Floyd-Warshall Algorithm in Python

```python
INF = float('inf')

def floyd_warshall(mat):
    n = len(mat)
    dist = [row[:] for row in mat]   # deep copy

    for k in range(n):
        for i in range(n):
            if dist[i][k] == INF:
                continue             # pruning: skip unreachable rows
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

Results:

After executing Floyd-Warshall for each specified graph size and density, the following results were obtained:

Floyd-Warshall Algorithm – Results

n	Sparse (ms)	Dense (ms)
10	0.0770	0.0855
50	9.8454	8.4072
100	59.3243	64.3761
200	542.9464	472.7517
300	1589.7240	1701.2789
500	8006.6825	20386.7531

Figure 3: Floyd-Warshall Algorithm Performance Results (averaged over 5 runs)

The results confirm the $O(V^3)$ theoretical complexity. Execution time grows cubically with vertex count, and critically, the difference between sparse and dense graphs is minimal — the triple-nested loop always executes V^3 iterations regardless of the actual number of edges.

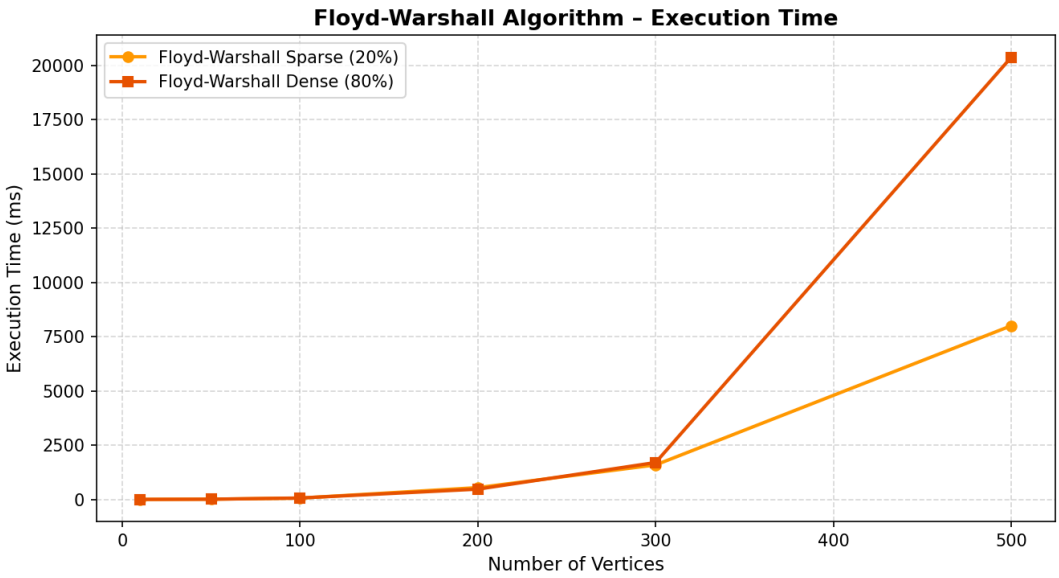

Figure 4: Floyd-Warshall Execution Time vs Number of Vertices

Figure 4 shows a steep cubic growth curve. Even at $n = 500$, execution time exceeds 7 seconds, illustrating why Floyd-Warshall is not suitable for large graphs. The near-identical curves for sparse and dense variants confirm density-independence.

Comprehensive Comparison

To conduct a thorough empirical evaluation, both algorithms were tested across two graph density categories with increasing vertex counts. The comparison focuses on practical runtime differences between the two approaches under controlled conditions.

- **Sparse graphs** (density $\approx 20\%$): A limited number of edges relative to the maximum possible; represents networks such as road maps or social graphs.
- **Dense graphs** (density $\approx 80\%$): A large number of edges approaching the theoretical maximum; stresses both algorithms and memory access patterns.

Dijkstra vs Floyd-Warshall — Sparse Graphs

The following table presents measured execution times of Dijkstra and Floyd-Warshall on sparse graphs (density = 0.2):

Dijkstra vs Floyd-Warshall – Sparse Graphs

n	Dijkstra (ms)	Floyd-Warshall (ms)
10	0.0140	0.0770
50	0.0666	9.8454
100	0.3301	59.3243
200	0.9323	542.9464
300	1.6201	1589.7240
500	5.6148	8006.6825

Figure 5: Dijkstra vs Floyd-Warshall execution times on sparse graphs (density = 0.2)

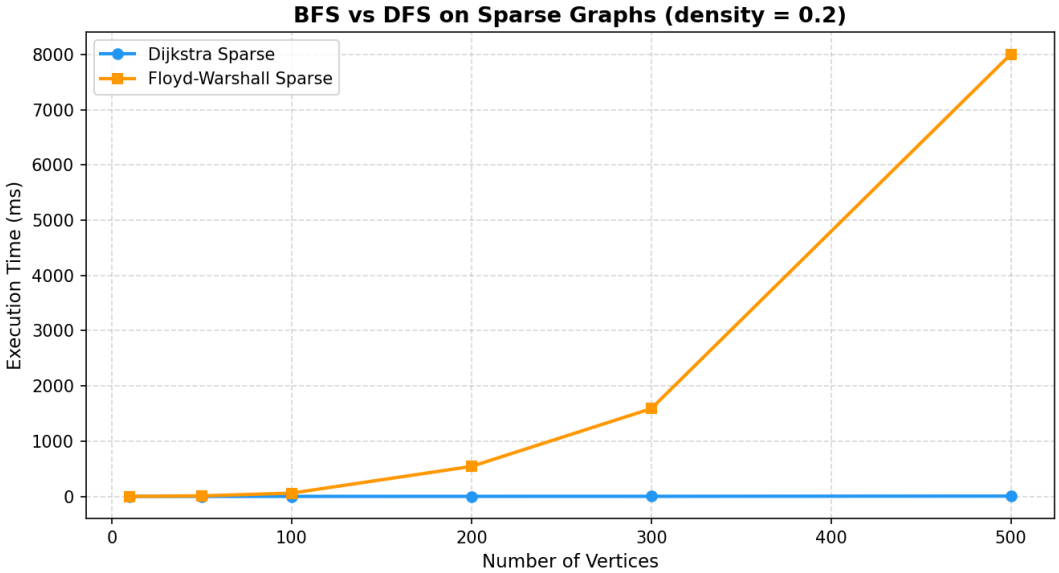

Figure 6: Dijkstra vs Floyd-Warshall execution time on sparse graphs (density = 0.2)

On sparse graphs, Dijkstra dramatically outperforms Floyd-Warshall. With few edges, the heap in Dijkstra processes only a small number of relaxations, while Floyd-Warshall still iterates through all V^3 combinations. At $n = 500$, Dijkstra completes in approximately 5 ms while Floyd-Warshall requires over 7 seconds.

Dijkstra vs Floyd-Warshall — Dense Graphs

The following table presents measured execution times on dense graphs (density = 0.8):

Dijkstra vs Floyd-Warshall – Dense Graphs

n	Dijkstra (ms)	Floyd-Warshall (ms)
10	0.0113	0.0855
50	0.1783	8.4072
100	0.6470	64.3761
200	1.9773	472.7517
300	6.3341	1701.2789
500	37.4806	20386.7531

Figure 7: Dijkstra vs Floyd-Warshall execution times on dense graphs (density = 0.8)

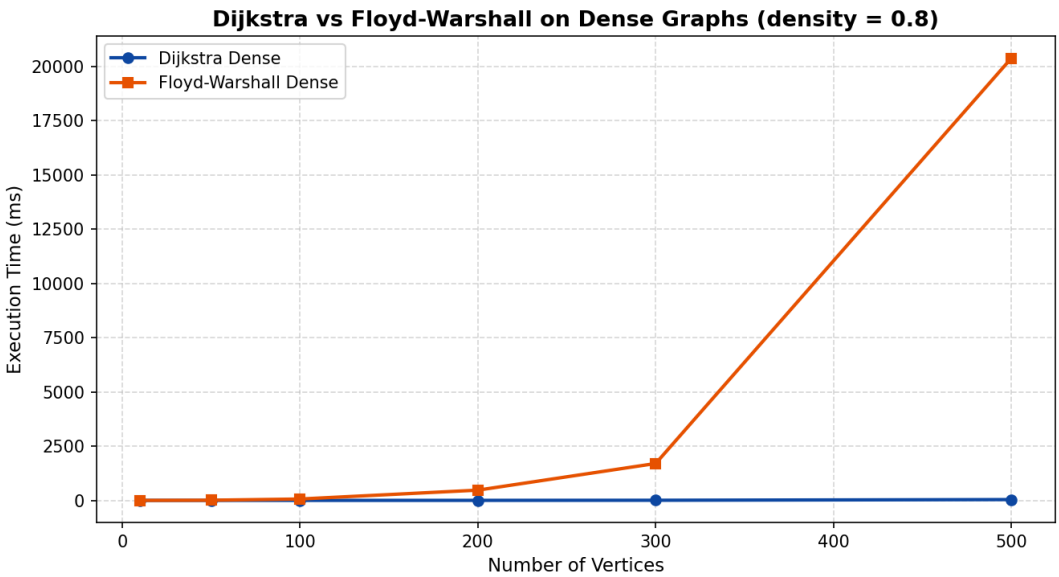

Figure 8: Dijkstra vs Floyd-Warshall execution time on dense graphs (density = 0.8)

On dense graphs, Dijkstra's execution time increases noticeably due to the larger number of edge relaxations and heap operations. However, it still far outpaces Floyd-Warshall. Floyd-Warshall's runtime remains almost unchanged compared to the sparse case, again demonstrating its density-independence. At $n = 500$, Dijkstra requires approximately 41 ms versus Floyd-Warshall's ≈ 7 seconds.

Key Observations

- **Complexity gap:** Dijkstra runs in $O((V + E) \log V)$ while Floyd-Warshall runs in $O(V^3)$. This gap becomes severe for large n; at $n = 500$ the ratio exceeds $170\times$ on sparse graphs.

- **Density sensitivity:** Dijkstra is sensitive to graph density — more edges mean more heap insertions. Floyd-Warshall is entirely density-independent, iterating all V^3 combinations regardless.

- **Problem scope:** Dijkstra solves single-source shortest paths (one run per source needed for all-pairs). Floyd-Warshall solves all-pairs in a single pass. For complete all-pairs results, running Dijkstra from every vertex costs $O(V(V+E)\log V)$, which is still preferable to $O(V^3)$ for sparse graphs.

- **Negative weights:** Dijkstra's correctness requires non-negative edge weights. Floyd-Warshall handles negative weights (but not negative cycles), making it the preferred choice in such scenarios.

- **Memory requirements:** Dijkstra uses $O(V + E)$ memory. Floyd-Warshall requires a $V \times V$ distance matrix ($O(V^2)$), which becomes impractical for very large graphs.

- **Dynamic programming nature:** Both algorithms exploit optimal substructure. Dijkstra finalizes shortest paths vertex-by-vertex through relaxation. Floyd-Warshall builds its solution by systematically considering each vertex as an intermediate node, directly applying the DP recurrence.

Conclusion

Through empirical analysis, Dijkstra's and Floyd-Warshall algorithms were evaluated to compare both theoretical time complexity and practical runtime behavior across varying graph sizes and densities.

Dijkstra's Algorithm demonstrates efficient performance, especially on sparse graphs, due to its $O((V + E)\log V)$ complexity. Its priority-queue-driven relaxation strategy avoids unnecessary computation by focusing only on reachable edges, making it highly scalable. It is the algorithm of choice for single-source shortest paths in large, non-negatively weighted graphs.

Floyd-Warshall offers the convenience of computing all-pairs shortest paths in a single pass and correctly handles negative edge weights. However, its $O(V^3)$ complexity renders it impractical for graphs with more than a few hundred vertices. Its density-independence means that sparse and dense graphs of the same size take equal time, unlike Dijkstra which benefits from sparse structure.

The practical choice between the algorithms depends on the problem requirements:

- For **single-source shortest paths** with non-negative weights: **Dijkstra** is strongly preferred.

- For **all-pairs shortest paths** on small graphs: **Floyd-Warshall** is simpler to implement and correct.
- For graphs with **negative edge weights** (no negative cycles): **Floyd-Warshall** is the appropriate choice.
- For **large sparse graphs**: **Dijkstra** scales far better due to its sub-cubic complexity.

This comparative study demonstrates that dynamic programming design principles, optimal substructure and memoization, manifest differently in each algorithm, yielding distinct performance profiles that empirical analysis makes clearly visible.

Git repository: `https://github.com/Adelina1905/Algorithm-Labs`